

Scale and Load Balance Your Architecture

Elastic Load Balancing | EC2 Auto Scaling | Launch Templates | CloudWatch Alarms

Course: CYB 222 – Linux Systems Administration & Security (AWS Cloud Module) **Author:** Atif Memon

Portfolio: github.com/atifkaloodi1/cloud-security-labs **Profile:** gravatar.com/atifmemon

✓ **LAB RESULT:** 35 / 35 — full marks on all six tasks (AMI, Load Balancer, Launch Template, Auto Scaling Group, LB check, Auto Scaling check, Web Server 1 termination).

Lab Overview & Objectives

AWS Academy — Scale and Load Balance Your Architecture • Duration ~30 min

This lab uses Elastic Load Balancing (ELB) and EC2 Auto Scaling to make a single-server web application highly available and elastic. Traffic is spread across multiple EC2 instances in two Availability Zones, and capacity is added or removed automatically in response to CPU load measured by Amazon CloudWatch.

Objectives

- Create an Amazon Machine Image (AMI) from a running instance.
- Create an Application Load Balancer with a target group.
- Create a launch template and an Auto Scaling group.
- Automatically scale instances in and out based on demand.
- Create CloudWatch alarms and monitor infrastructure performance.

CAUTION

Never run 20 or more concurrent instances in this lab environment (any size). Doing so triggers immediate deactivation of the AWS account and deletion of all resources. The Auto Scaling group here is capped at a maximum of 6.

Architecture

From a single web server to an elastic, load-balanced fleet

Starting state

A single Web Server 1 lives in Public Subnet 2 (AZ-B). A Multi-AZ RDS database (primary + secondary) sits in the private subnets, with a NAT gateway in Public Subnet 1 providing outbound access. There is a single point of failure at the web tier.

Scenario

You start with the following infrastructure:

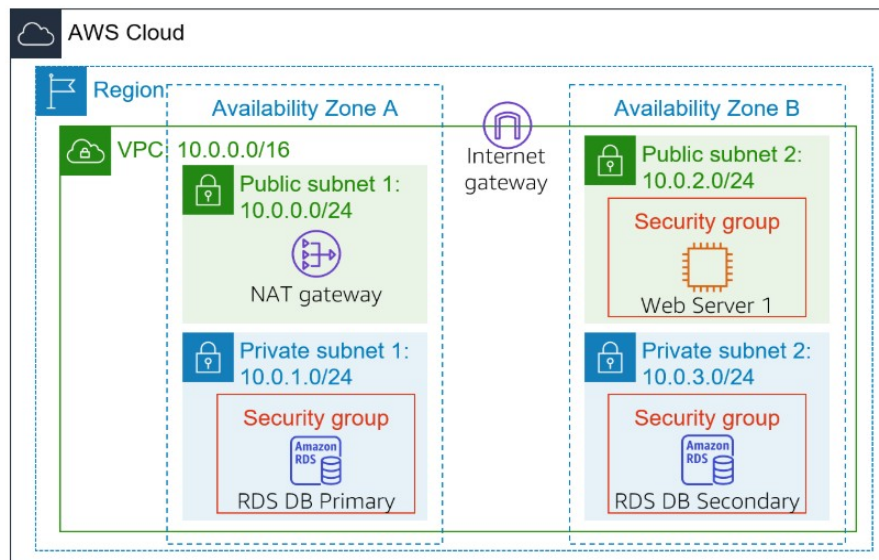


Figure 1 — Starting infrastructure: one web server, no load balancing.

Final state

An internet-facing Application Load Balancer spans both public subnets and distributes requests to an Auto Scaling group of web instances running across both AZs. The single point of failure is removed and the web tier now scales with demand.

The final state of the infrastructure is:

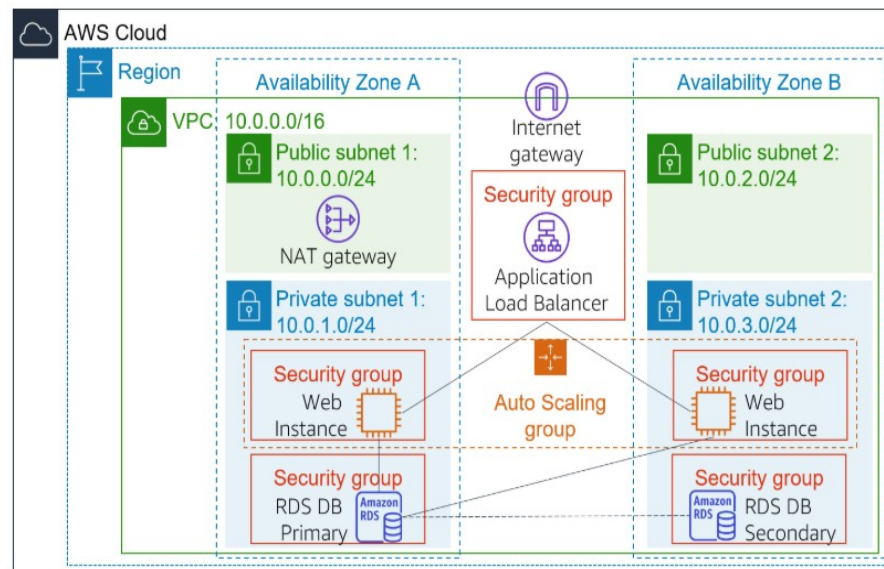


Figure 2 — Final infrastructure: ALB + Auto Scaling group across two AZs.

SECURITY INSIGHT

This is the reference pattern for web-tier high availability on AWS: an ALB in the public subnets fronting a private, auto-scaled compute fleet. The instances themselves are never exposed directly to the internet — only the load balancer's security group is — which shrinks the attack surface while keeping the tier resilient to instance or AZ failure.

Task 1 — Create an AMI for Auto Scaling

Capture the boot disk of Web Server 1 so identical instances can be launched

1. Open EC2 → Instances. Confirm **Web Server 1** shows **2/2 checks passed**.
2. Select **Web Server 1**, then **Actions** → **Image and templates** → **Create image**.
3. Set the values exactly (case-sensitive):

Create image

Image name: WebServerAMI
Image description: Lab AMI for Web Server

4. Choose **Create image**. A banner confirms the new AMI ID — this AMI is used by the launch template in Task 3.

NOTE

An AMI is a point-in-time snapshot of the root volume plus launch metadata. Because every scaled instance boots from the same image, the fleet is immutable and reproducible — no manual per-instance configuration drift.

Instances (1/1) info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Find Instance by attribute or tag (case-sensitive)

Name: Web Server 1 Clear filters

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
Web Server 1	i-0362b56daa6adeaca	Running	t2.micro	2/2 checks passed	us-east-1a	-	13.216.214.255	-	-

i-0362b56daa6adeaca (Web Server 1)

Details Status and alarms Monitoring Security Networking Storage Tags

Instance summary info

Instance ID i-0362b56daa6adeaca	Public IPv4 address 13.216.214.255 open address	Private IPv4 addresses 10.0.0.75
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-0-75.ec2.internal	Private IP DNS name (IPv4 only) ip-10-0-0-75.ec2.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t2.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 13.216.214.255 (Public IP)	VPC ID vpc-0af043bb850f97f97 (Lab VPC)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0c7e62efc3f47d7e (Public Subnet 1)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:625892102011:instance/i-0362b56daa6adeaca	

Web Server 1 running with 2/2 status checks passed — t2.micro in us-east-1a, in Lab VPC / Public Subnet 1.

Currently creating AMI ami-0e0df51a1a8e42119 from instance i-0362b56daa6adeaca. Check that the AMI status is 'Available' before deleting the instance or carrying out other actions related to this AMI.

Instances (1) info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Find Instance by attribute or tag (case-sensitive)

Name: Web Server 1 Clear filters

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
Web Server 1	i-0362b56daa6adeaca	Running	t2.micro	2/2 checks passed	us-east-1a	-	13.216.214.255	-	-

Confirmation banner: AMI ami-0e0df51a1a8e42119 (WebServerAMI) being created from instance i-0362b56daa6adeaca.

Task 2 — Create a Load Balancer

Target group + internet-facing Application Load Balancer

2a — Target group

1. EC2 → Target Groups → Create target group.

2. Target type **Instances**; name **LabGroup**; VPC **Lab VPC**.
3. Skip target registration (no instances yet) → **Create target group**.

NOTE

Target groups define where the ALB forwards traffic. An ALB operates at Layer 7 (the request level) and can route to different target groups based on the request URL — for example, sending /api or mobile traffic to a separate fleet.

Successfully created the target group: **LabGroup**. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the **Targets** tab.

LabGroup Actions

Details

arn:aws:elasticloadbalancing:us-east-1:625892102011:targetgroup/LabGroup/716e4ad01c674bb0

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-0af043bb850f97f97
IP address type IPv4	Load balancer None associated		

0 Total targets	0 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
--------------------	--------------	----------------	-------------	--------------	---------------

0 Anomalous

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (0) Info Anomaly mitigation: Not applicable Deregister Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch time
No registered targets								

You have not registered targets to this group yet

[Register targets](#)

LabGroup target group created — target type Instance, HTTP:80, in Lab VPC, 0 registered targets (none yet), no load balancer associated.

2b — Application Load Balancer

1. EC2 → **Load Balancers** → **Create load balancer** → **Application Load Balancer**.
2. Name **LabELB**; scheme internet-facing.
3. Network mapping — VPC **Lab VPC**; map **Public Subnet 1** (AZ-A) and **Public Subnet 2** (AZ-B).
4. Security groups — select **Web Security Group** and remove the **default** group.
5. Listener **HTTP:80** → default action forward to **LabGroup**.
6. Create load balancer. State will show **provisioning** — no need to wait.

SECURITY INSIGHT

Spanning two AZs is what buys the availability: if an entire Availability Zone degrades, the ALB keeps serving from the healthy zone. Only the ALB's security group faces the internet; the instances stay private.

✔ Successfully created load balancer: **LabELB**
It might take a few minutes for your load balancer to fully set up and route traffic. Targets will also take a few minutes to complete the registration process and pass initial health checks.

🔔 Introducing configurable log deliveries for ALB
Send Application Load Balancer logs to multiple destinations — CloudWatch, S3, or Kinesis Firehose — giving you greater flexibility in how you store, analyze, and manage logs. Visit [Logging in the Integrations tab](#).

LabELB 🔄 Actions

▼ Details			
Load balancer type Application	Status 🔄 Provisioning	VPC vpc-0af043bb850f97f97	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z355XDOTRQ7X7K	Availability Zones subnet-040d0f05595b01ac us-east-1b (use1-az1) subnet-0c7e62efc3f47d7e1 us-east-1a (use1-az2)	Date created August 11, 2026, 13:40 (UTC-04:00)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:625892102011:loadbalancer/app/LabELB/a4cc41b27edcbd62		DNS name Info LabELB-1997750299.us-east-1.elb.amazonaws.com (A Record)	

[Listeners and rules](#) | [Network mapping](#) | [Resource map](#) | [Security](#) | [Monitoring](#) | [Integrations](#) | [Attributes](#) | [Capacity](#) | [Tags](#)

Listeners and rules (1) [Info](#) 🔄 Manage rules 🔄 Manage listener ➕ Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

☐ Protocol:Port

▼

Default action

▼

Rules

▼

ARN

▼

Security policy

▼

Default SSL/TLS certificate

▼

mTLS

▼

Trust store

• Forward to target group

LabELB created — Application load balancer, scheme Internet-facing, spanning two AZs (us-east-1a / us-east-1b) in Lab VPC. DNS: LabELB-1997750299.us-east-1.elb.amazonaws.com.

Task 3 — Launch Template & Auto Scaling Group

The blueprint for scaled instances, plus the group that manages them

3a — Launch template

1. EC2 → Launch Templates → Create launch template.
2. Configure:

LabConfig — launch template

```
Launch template name: LabConfig
Auto Scaling guidance: Provide guidance... [enabled]
AMI (My AMIs):       Web Server AMI (WebServerAMI)
Instance type:       t2.micro
Key pair name:       vockey
Security group:      Web Security Group
Advanced > Detailed CloudWatch monitoring: Enable
```

3. Create launch template.

NOTE

Detailed CloudWatch monitoring reports metrics at 1-minute intervals instead of 5. Auto Scaling reacts faster to load changes as a result — important for a demo where scaling must happen within minutes.

Success
Successfully created LabConfig(lt-0c69f011d399c65e).

Actions log

Next Steps

Launch an instance

With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.

[Launch instance from this template](#)

Create an Auto Scaling group from your template

Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.

[Create Auto Scaling group](#)

Create Spot Fleet

A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance (of each instance type in each Availability Zone) is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks.

[Create Spot Fleet](#)

[View launch templates](#)

LabConfig launch template created (lt-0c69f011d399c65e) — the blueprint the Auto Scaling group uses to launch identical instances.

3b — Auto Scaling group

1. From the launch template, **Actions** → **Create Auto Scaling group**.
2. Name **Lab Auto Scaling Group**; template **LabConfig**.
3. Network — VPC **Lab VPC**; subnets **Private Subnet 1** and **Private Subnet 2**.
4. Attach to existing load balancer → target group **LabGroup**; enable **group metrics collection** in **CloudWatch**.
5. Group size and scaling policy:

Group size & LabScalingPolicy

```
Desired capacity: 2
Minimum capacity: 2
Maximum capacity: 6

Scaling policy: Target tracking
Policy name:    LabScalingPolicy
Metric type:    Average CPU Utilization
```

Target value: 60

6. Add tag **Name = Lab Instance** (propagates to launched instances) → review → **Create Auto Scaling group**.

SECURITY INSIGHT

Target tracking is a closed control loop: you declare the goal (keep average CPU near 60%) and Auto Scaling adds or removes instances to hold it there, within the 2–6 bound. You never write threshold math — CloudWatch creates the alarms for you. Instances launch into the private subnets, so the fleet scales without ever gaining a public IP.

Lab Auto Scaling Group, 1 Scaling policy created successfully. Group metrics collection is enabled.

Auto Scaling groups (1) Info

Search your Auto Scaling groups

Launch configurationsLaunch templatesActionsCreate Auto Scaling group

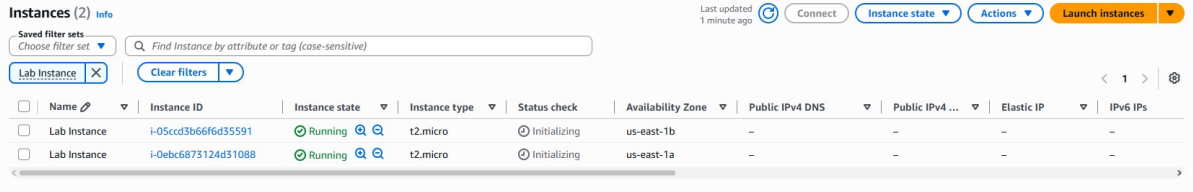
Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max	Availability Zones	Creation time
Lab Auto Scaling Group	At desired capacity	LabConfig Version Default	2	2/2 Healthy	2	2	6	2 Availability Zones	Tue Aug 11 2026 1...

Lab Auto Scaling Group created — At desired capacity, using LabConfig, 2 instances 2/2 healthy across 2 AZs (desired 2 / min 2 / max 6), 1 scaling policy, group metrics enabled.

Task 4 — Verify Load Balancing

Confirm healthy targets and reach the app through the ALB

1. EC2 → **Instances** — two new **Lab Instance** instances should appear.



Two Lab Instance instances launched by Auto Scaling — one in us-east-1a and one in us-east-1b, both t2.micro (status checks still initializing).

2. EC2 → **Target Groups** → **LabGroup** → **Targets** tab. Wait until both show **healthy**.

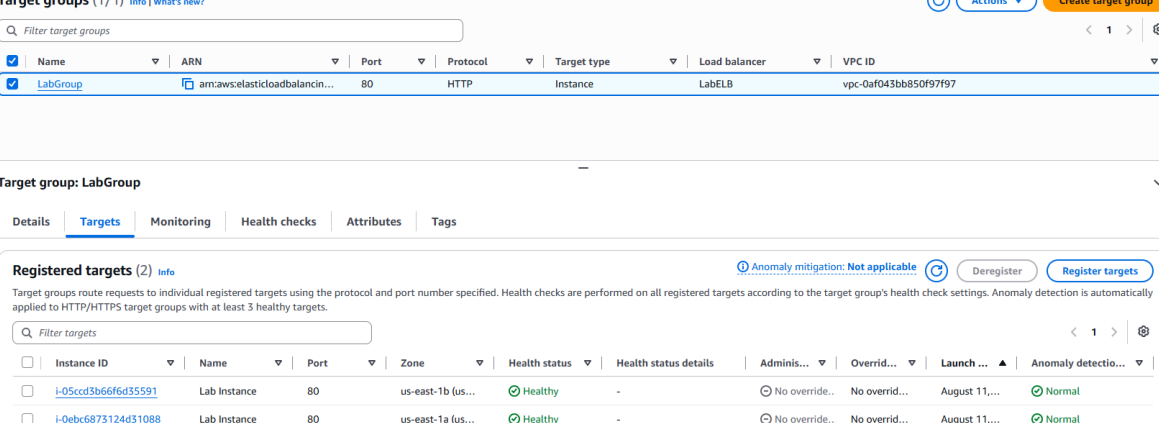
3. Load Balancers → **LabELB** → copy the **DNS name** (omit "(A Record)").

```
# ALB DNS name — paste into a new browser tab
LabELB-1997750299.us-east-1.elb.amazonaws.com
```

4. Open the DNS name in a browser — the application renders, proving the ALB forwarded the request to a backend instance and returned the response.

NOTE

“Healthy” means the instance passed the load balancer's health check, so the ALB is willing to route traffic to it. Unhealthy targets are automatically taken out of rotation — a failed instance simply stops receiving requests.

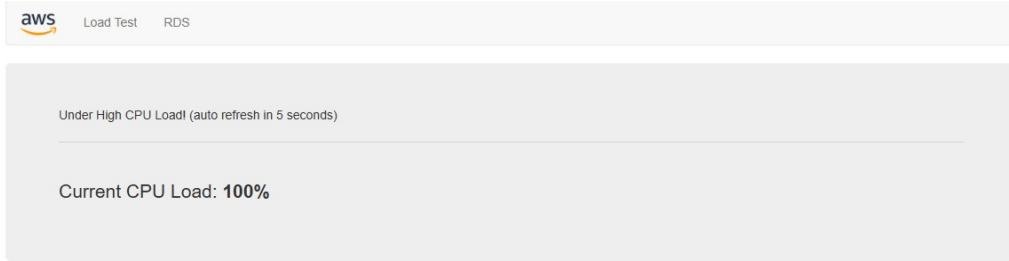


Target group: LabGroup

Registered targets (2)

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...	Overrid...	Launch ...	Anomaly detectio...
i-05ccd3b66f6d35591	Lab Instance	80	us-east-1b (us...	Healthy	-	No override...	No overrid...	August 11,...	Normal
i-0ebc6873124d31088	Lab Instance	80	us-east-1a (us...	Healthy	-	No override...	No overrid...	August 11,...	Normal

LabGroup Targets tab — both Lab Instances registered on HTTP:80 and showing Health status = Healthy (us-east-1a and us-east-1b), associated with LabELB.



The web application served through the LabELB DNS name — the ALB received the request and returned a response from a backend instance. Shown here under the Load Test with Current CPU Load at 100%.

Task 5 — Test Auto Scaling

Drive CPU up and watch the fleet grow

1. CloudWatch → **All alarms**. Two alarms were auto-created by the ASG (an **AlarmHigh** and an **AlarmLow**).
2. Open the **AlarmHigh** alarm (state **OK**). It fires when average CPU exceeds the target for a sustained period.

Name	State	Actions	Actions muted - new	Last state update (UTC)	Conditions
TargetTracking-Lab Auto Scaling Group-AlarmLow-990a19b6-7a4e-452a-874a-d75db5d666cb	In alarm	Actions enabled	-	2026-08-11 18:14:56	CPUUtilization < 42 for 15 datapoints within 15 minutes
TargetTracking-Lab Auto Scaling Group-AlarmHigh-4240b688-d2ed-4b91-a85e-612a5c9744f2	OK	Actions enabled	-	2026-08-11 18:01:00	CPUUtilization > 60 for 3 datapoints within 3 minutes

The two alarms auto-created by the ASG — AlarmHigh (CPUUtilization > 60 for 3 datapoints, currently OK) scales out; AlarmLow (currently In alarm) scales in. Together they hold average CPU near the target.

3. In the app browser tab, choose **Load Test** next to the AWS logo. The page auto-refreshes so every instance generates load.
4. Back in CloudWatch (refresh ~every 60s): within ~5 min **AlarmLow** → OK and **AlarmHigh** → **In alarm** once CPU holds above 60% for ~3 min.
5. EC2 → **Instances** — more than two **Lab Instance** instances are now running, launched in response to the alarm.

SECURITY INSIGHT

The optional tweak of lowering the target value to 50 in LabScalingPolicy makes the group more sensitive, so the scale-out event is easier to observe in the short lab window. In production you'd tune this against real traffic and cost targets rather than for speed of demonstration.

Dynamic scaling policy created or edited successfully.

Auto Scaling groups (1/1) Info

Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max	Availability
Lab Auto Scaling Group	At desired capacity	LabConfig Version Default	2	2/2 Healthy	2	2	6	2 Available

Auto Scaling group: Lab Auto Scaling Group

Details | Integrations | **Automatic scaling** | Instance management | Instance refresh | Activity | Monitoring | Tags

Scaling policies resize your Auto Scaling group to meet changes in demand. With reactive dynamic scaling policies, you can track specific CloudWatch metrics and take action when the CloudWatch alarm threshold is met. Use predictive scaling policies along with dynamic scaling policies in the following situations: when your application demand changes quickly, but with a recurring pattern, or when your EC2 instances require more time to initialize.

Dynamic scaling policies (1) Info [Actions](#) [Create dynamic scaling policy](#)

LabScalingPolicy

Policy type
Target tracking scaling

Enabled or disabled
Enabled

LabScalingPolicy (target tracking, Enabled) on the ASG's Automatic scaling tab — “Dynamic scaling policy created or edited successfully” after lowering the target value to make scale-out fire sooner.

NOTE

AlarmHigh entered the **In alarm** state once average CPU held above the target for three consecutive datapoints — the scale-out event below (a new Lab Instance initializing) is the direct result of that alarm firing and is captured in the following screenshot.

Successfully initiated termination (deletion) of i-0362b56daa6adeaca

Instances (1/6) Info

Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
Lab Instance	i-035a9b47ef9ae8c0	Running	t2.micro	Initializing	us-east-1a	-	-	-	-
Web Server 1	i-0362b56daa6adeaca	Shutting-d...	t2.micro	-	us-east-1a	-	13.216.214.255	-	-
Lab Instance	i-05ccd3b66fd35591	Running	t2.micro	2/2 checks pass	us-east-1b	-	-	-	-
Bastion Host	i-0c874a589fb5b445c	Running	t2.micro	2/2 checks pass	us-east-1a	-	107.21.91.73	-	-
Lab Instance	i-0ebc6873124d31088	Running	t2.micro	2/2 checks pass	us-east-1a	-	-	-	-

Scale-out fired — a new Lab Instance (i-035a9b47...) is Initializing, taking the fleet above the original two (6 instances total in the account). Auto Scaling launched it in response to AlarmHigh. (Web Server 1 is shutting down here as part of Task 6.)

Task 6 — Terminate Web Server 1

Retire the original single server

- EC2 → **Instances** → select **Web Server 1** only.
- Instance state → **Terminate instance** → **Terminate**.

NOTE

Web Server 1 only existed to seed the AMI. Once the Auto Scaling group is serving traffic, the original instance is redundant — the fleet is now defined entirely by the launch template and scaling policy, not by any hand-built server.

Successfully initiated termination (deletion) of i-0362b56daa6adeaca

Instances (1/6) Info

Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets Choose filter set Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
Lab Instance	i-035a9b47ef9ae8c0	Running	t2.micro	Initializing	us-east-1a	-	-	-	-
Web Server 1	i-0362b56daa6adeaca	Shutting-d...	t2.micro	-	us-east-1a	-	13.216.214.255	-	-
Lab Instance	i-05ccd3b66fd35591	Running	t2.micro	2/2 checks pass	us-east-1b	-	-	-	-
Bastion Host	i-0c874a589fb5b445c	Running	t2.micro	2/2 checks pass	us-east-1a	-	107.21.91.73	-	-
Lab Instance	i-0ebc6873124d31088	Running	t2.micro	2/2 checks pass	us-east-1a	-	-	-	-

“Successfully initiated termination of i-0362b56daa6adeaca” — Web Server 1 is Shutting-down. The fleet now runs entirely from the LabConfig launch template and LabScalingPolicy; the seed instance is no longer needed.

Configuration Reference

Exact, case-sensitive names used by the grading script

Resource	Type	Name / Value (exact)
AMI	Machine image	WebServerAMI
Target group	ELB target group	LabGroup
Load balancer	Application LB	LabELB
Launch template	Launch template	LabConfig
Instance type	EC2 size	t2.micro
Key pair	SSH key	vockey
Auto Scaling group	ASG	Lab Auto Scaling Group
Scaling policy	Target tracking	LabScalingPolicy
Target CPU	Metric target	60% avg CPU
Capacity	min / desired / max	2 / 2 / 6
Instance tag	Propagated tag	Name = Lab Instance

Security Insights & Key Takeaways

Notes for the CYB 222 capstone article

- **High availability by design.** An ALB across two AZs fronting an auto-scaled fleet removes the single point of failure at the web tier. Instance and AZ failures become non-events.
- **Reduced attack surface.** Scaled instances live in private subnets and never receive a public IP; only the load balancer's security group is internet-facing. Compute is isolated from direct inbound traffic.
- **Immutable infrastructure.** Every instance boots from the same AMI via the launch template — no configuration drift, no snowflake servers, and a clean rebuild path if an instance is compromised: terminate and let Auto Scaling replace it.
- **Elasticity as a control.** Target-tracking scaling is a closed feedback loop that also blunts certain load-based abuse: capacity absorbs spikes (up to the max), while the max cap prevents runaway cost from a flood.
- **Health checks enforce integrity.** Unhealthy targets are automatically removed from rotation, so a failing or degraded instance stops serving traffic without manual intervention.
- **Observability built in.** CloudWatch detailed monitoring + group metrics give per-minute visibility into the fleet — the same telemetry a SOC would use to spot anomalous CPU or scaling behaviour.

Lab Result

AWS Academy submission report

All six tasks were graded and passed with a perfect score.

Total score	35/35
Task 1 - AMI created	5/5
Task 2 - Load Balancer created	5/5
Task 3a - Launch Template created	5/5
Task 3b - Auto Scaling Group created	5/5
Task 4 - Load Balancer check	5/5
Task 5 - Auto Scaling check	5/5
Task 6 - Web Server 1	5/5

Submission report — Total score 35 / 35.